

Phase 9 – Cloudy Day Test Feedback

Testers: Group 8

Target Group: Group 12 – FCUL Anime API

Date: June 3, 2026

API tested: <http://34.62.61.34>

Smoke and stress tests were conducted against Group 12's deployed API during the Cloudy Day window (16:00–19:00) by four testers:

Tester	Stress Test	Smoke Test
Joana Carrasqueira	GET /anime	PUT /users/{user_id}
Leonor Silva	GET /users	POST /users
Tiago Pereira	GET /rating	POST /rating
Tiago Pina	GET /anime/search	DELETE /rating/{id}

Joana Carrasqueira

Documentation Feedback

The Swagger documentation for Group 12's API is well structured, with a clear separation between the Catalog, Users, and Ratings services. The use of reusable schemas (`Anime`, `User`, `Rating`, `RatingResponse`, etc.) and consistent endpoint summaries makes the API easy to understand.

That said, a few improvements could be made. The `username` field in `UserUpdate` has no documented constraints — no `minLength`, `maxLength`, or format validation — which led to bugs detected during testing, such as the 255-character database limit not being surfaced to the client. These should be added to the documentation. The `user_id` path parameter is typed as `integer` but has no `minimum` defined, allowing negative values to pass validation silently.

Stress test: GET /anime

The stress test was executed with Locust against the `GET /anime` endpoint, testing all 50 combinations of `limit` ∈ {1, 5, 10, 15, 20} and `offset` ∈ {0, 20, 40, 60, 80, 100, 120, 140, 160, 180}, with 50 concurrent users spawned at 5 users/s, a wait time of 0.1–0.5 s between requests, and a total duration of 60 s.

The test handled 2882 requests with 0 failures, sustaining a peak RPS of ~67.3 and a steady-state RPS of ~65–66. Median response time was 440 ms, average 459 ms, with a p95 of 630 ms and p99 of 740 ms. The single maximum of 1015 ms (limit=5, offset=120) was an isolated outlier.

Response times were consistent across all `limit`/`offset` combinations (median 430–490 ms), showing pagination parameters have no meaningful effect on performance. Payload size scaled predictably with `limit` — from ~75 B at limit=1 to ~1630 B at limit=20 — without noticeably increasing latency, suggesting the bottleneck is network/serialization rather than query time.

Smoke test: `PUT /users/{user_id}`

The smoke test was performed using Postman on the `PUT /users/{user_id}` endpoint, covering functional, path parameter, input validation, and SQL injection scenarios. User existence was verified via `GET /users/{user_id}` before and after each test.

The endpoint works correctly for users already present in the database from previous sessions — not-found, null, empty body, and empty string cases all return clear, consistent error messages. However, several issues were found.

Updating a user that was created during the same test session returned `500 Internal Server Error`, while the same operation on a pre-existing user succeeded with `200`. Type validation (integer, boolean, array as username) and malformed JSON are correctly rejected with `422`. Whitespace-only usernames (`" "`) are accepted and persisted. Usernames of 256 or more characters cause a `500` instead of a `422`, as the API has no documented length constraint and the crash occurs at the database layer. SQL injection is not possible — all tested payloads were stored as literal strings or rejected at the JSON parser, confirming parameterized queries are in use. XSS payloads are stored as literal strings, which is safe at the API layer but worth addressing if a frontend is introduced.

Observation

`GET /anime` sustained zero failures across 2882 requests, peaking at ~67 RPS with 50 concurrent users, a strong result for a GKE deployment. `PUT /users/{user_id}` is functional for pre-existing users but has a critical server-side bug when updating recently created users, and lacks server-side enforcement of the 255-character username limit, causing a `500` that should be a `422`.

Leonor Silva

Documentation feedback

The OpenAPI specification for Group 12's API is clearly organised into three services (Catalog, Users, Ratings) with reusable schemas and concise endpoint summaries, making it straightforward to navigate.

However, several gaps were identified. The `POST /users` endpoint only documents `201` and `500` as possible responses, omitting `422 Unprocessable Entity`, which is the actual response for all validation failures. The `ErrorResponse` schema defines `detail` as `type: string`, but in practice `422` responses return `detail` as an array of objects containing `type`, `loc`, `msg`, and `input` fields — making the schema incorrect. Additionally, `GET /users` has no `limit` or `offset` parameters documented, and the `UserCreate` schema lacks `minLength` and `maxLength` constraints on `username`.

Stress test: `GET /users`

The stress test was executed with Locust against the `GET /users` endpoint using a stepped load profile (`LoadTestShape`) progressing from 10 to 500 concurrent users over 5 phases of 60 seconds each, with zero wait time between requests. The test was run twice during the session.

Run 1 handled 13,978 requests with 0 failures, peaking at 70 avg RPS and 77 instantaneous RPS at 50 concurrent users. Run 2, performed later in the session after other groups had added more users to the database, handled 10,548 requests with 0 failures, peaking at only 51.9 avg RPS — a 26% drop despite identical configuration. The p50 response time at Phase 4 (250 users) worsened from

767 ms to 1,263 ms (+65%) between runs. In both runs, the sharpest degradation occurred between Phase 2 and Phase 3: p99 jumped approximately 8× (from ~1,155 ms to ~9,055 ms in Run 1). From Phase 3 onwards, adding more concurrent users provided no throughput gain, confirming the server had reached its processing ceiling at around 50–100 simultaneous connections.

Smoke test: `POST /users`

The smoke test was performed using Postman on the `POST /users` endpoint, covering functional, validation, boundary, and security-relevant scenarios.

The happy path worked correctly — valid users were created with `201` and unique IDs. However, several issues were found. The API accepts duplicate usernames without error, confirmed by `smoke_leonor_01` appearing four times in `GET /users` with different IDs (`508999026`, `497183876`, `282838035`, `66840616`). Empty strings (`""`) and whitespace-only strings (`" "`) are also accepted as valid usernames. When a username exceeds 255 characters, the server returns `500 Internal Server Error` with the raw database error exposed in the response body: `STRING(255) has maximum length 255 but got a value with length 1728`. No input sanitisation is applied — SQL injection and XSS payloads were accepted and stored verbatim, though neither is currently exploitable given the ORM usage and absence of a frontend.

Observation

`GET /users` sustained zero failures across 24,526 total requests but showed clear performance degradation above 50 concurrent users, compounded by unbounded payload growth due to missing default pagination. `POST /users` is functional for basic creation but lacks uniqueness enforcement, input length validation, and sanitisation, with the most critical issue being a `500` that leaks internal database schema details when usernames exceed 255 characters.

Tiago Pereira

Documentation feedback

The Swagger/OpenAPI documentation for Group 12's API is generally well structured and easy to navigate, with a clear separation between Catalog, Users, and Ratings. During testing, this made it straightforward to locate the relevant endpoints and understand the expected request and response formats.

However, a few issues reduced its clarity in practice. The API mixes `snake_case`, `camelCase`, and capitalised fields such as `Name` and `Ranked`, which makes the schema less uniform and slightly harder to read. The error handling is also not always documented in a way that matches the actual responses observed during testing, since some endpoints provide only very generic error descriptions. In addition, the POST and PUT endpoints would benefit from more concrete example payloads, as this would make manual testing faster and reduce ambiguity when constructing requests.

Overall, the documentation is usable and well organised, but it would benefit from more consistency and more practical examples.

Smoke test: `POST /rating`

The smoke test was performed using Postman on Group 12's `POST /rating` endpoint, and it failed consistently during testing. Several combinations of `user_id` and `anime_id` that were already confirmed as existing through other endpoints still returned `404 User not found`, preventing the creation of new ratings.

Examples tested:

- `user_id: 442573965`, `anime_id: 34348`
- `user_id: 508147556`, `anime_id: 38577`
- `user_id: 442573965`, `anime_id: 38577`

The selected `user_id` and `anime_id` values were validated successfully through `GET /users`, `GET /users/{id}`, and `GET /anime/{id}`.

Stress test: `GET /rating`

The stress test was executed with Locust against Group 12's `GET /rating` endpoint, using a progressive load profile and zero wait time between requests. The test was run three times, and all runs showed the same overall behaviour: high response times, low throughput, and `503 Service Unavailable` failures under load.

Across the three runs, the endpoint handled between `448` and `676` requests, with between `23` and `53` failures. Median response times ranged from `21000 ms` to `23000 ms`, average response times from `24723.06 ms` to `27880.82 ms`, and maximum response times from `163927.84 ms` to `176009.72 ms`. Requests per second ranged from `1.57` to `2.27`, confirming unstable performance under progressive load.

Observation

In summary, Group 12's `POST /rating` endpoint was not functional for rating creation, while `GET /rating` remained available under lighter load but degraded significantly under stress across all three Locust runs.

Tiago Pina

Smoke Testing Findings (`DELETE /rating/{id}`)

The goal was to test the deletion of a rating. To avoid disrupting existing data, first was created a temporary rating, which uncovered several critical system flaws:

- **Endpoint Malfunction (`POST: /rating`):** Attempts to create a new rating failed with a **404 Not Found** error claiming the user did not exist. However, a separate `GET` request confirmed the user was successfully created and present in the database. This points to severe database synchronisation or foreign key validation issues.
- **Concurrency Timeouts:** Attempts to fetch existing ratings (`GET: /rating`) resulted in consistent timeouts, likely due to other groups stress-testing the server simultaneously.
- **State Inconsistency (`DELETE: /rating/{id}`):** When attempting to delete an existing rating (ID `5`), the server initially returned a false **404 Not Found** (claiming the rating didn't exist), even

though a subsequent `GET` request confirmed it was still there.

- **Final Verdict:** While the deletion eventually succeeded after persistent retries (returning a **204 No Content**), the API is highly unstable. It suffers from broken record creation, vulnerability to timeouts, and inconsistent database resolution.

Stress Testing Findings (`GET /anime/search`)

To investigate the timeouts observed during smoke testing, two sustained load tests were executed using Locust (running for 180 seconds each) during a stabilised server window.

Test Parameters & Reliability

- **Test 1 (200 Concurrent Users):** Achieved an average throughput of ~54.9 req/s with a 99.99% success rate (1 failure).
- **Test 2 (500 Concurrent Users):** Throughput dropped to ~37.3 req/s, with a 99.88% success rate (8 failures).

Latency Degradation

As concurrency increased from 200 to 500 users, response times degraded catastrophically rather than the application crashing outright:

- **Median Response (p50):** Spiked from 590 ms (Test 1) to **5.9 seconds** (Test 2).
- **95th Percentile (p95):** Spiked from 2.1 seconds (Test 1) to **39 seconds** (Test 2).
- **Maximum Response:** Reached up to ~101 seconds in Test 1 and **~155 seconds** in Test 2.

Error Analysis

Only 9 errors were recorded across both tests. All were **503 Service Unavailable** errors triggered by the gateway severing the connection due to upstream timeouts.

Core Conclusions

1. **Resource Starvation:** The server does not effectively shed excess load. Instead of throwing internal exceptions under pressure, it indefinitely queues requests, causing extreme latency spikes.
2. **Concurrency Bottlenecks:** The system handles 200 users adequately, but pushing to 500 users causes a total collapse in throughput and exponentially higher response times. This strongly suggests thread pool exhaustion or database connection limits.
3. **Gateway Disconnects:** The massive request queues eventually force the API gateway to terminate connections (resulting in 503 errors). This behaviour directly explains the constant timeouts experienced during the earlier smoke testing phase.